



南方科技大学

LAB 8: Introduction to AI Infrastructure

COE 201 - AI Large Models, Spring 2026

2026-05-17

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

- LLM Infrastructure: The Systems Bottleneck 1
- The GPU Execution Model 4
- Programming in Triton 11
- Advanced Operator Fusion with Triton 17
- Lab 8 Exercises 21

LLM Infrastructure: The Systems Bottleneck

Why AI scaling requires us to go deeper than PyTorch.

Scaling AI models requires engineering at two fundamentally different levels.

Macro: The Cluster Level



Distributing across thousands of Nodes.

- **Context:** How do nodes talk to each other?
- **Focus:** Parallelism (Data/Tensor/Pipeline).

Micro: The Device Level



Maximize speed for a single chip.

- **Context:** How do cores talk to Memory?
- **Focus:** Operator Fusion.

*Macro-level scalability is built on the foundation of **Single-Device Efficiency**,
we focus on the latter in this lab.*

Efficiency Barriers: The Two Walls



南方科技大学

1. Compute-Bound

Limit: Math Speed (TFLOPS)

Status: Cores at 100% capacity

Example: Large MatMuls

2. Memory-Bound

Limit: Memory Bandwidth

Status: Cores are **starving** (idle)

Example: Softmax, ReLU, LN

LLM inference is almost exclusively bottlenecked by the Memory Wall.

Example of the Memory Wall: Quick GeLU

$$y = x \cdot \sigma(1.702 \cdot x)$$

```
1 # 1. Compute sigma(1.702 * x), save to DRAM as temp_x
2 # 2. Load x and temp_x from DRAM, multiply
3 y = x * torch.nn.functional.sigmoid(1.702 * x)
```

Python

Solution: Operator Fusion

Custom GPU kernels compute everything in faster on-chip memory (SRAM), entirely bypassing slow memory (HBM)!

The GPU Execution Model

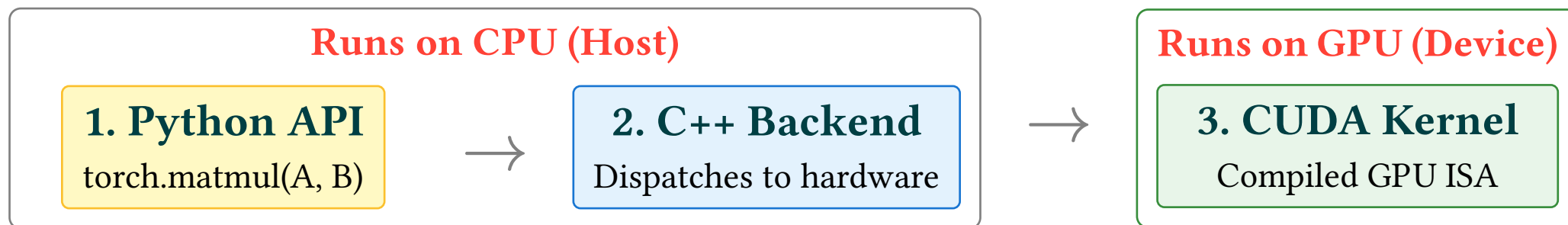
How hardware actually processes your tensor.

The Compute Engine: CPU vs GPU



Feature	CPU (Host)	GPU (Device)
Architecture	Complex ALUs + Large Cache	Simple ALUs + High Bandwidth
Core Count	Few (e.g., 64), deeply optimized	Thousands (e.g., 16k), dense
Efficiency	Low Latency (Sequential)	High Throughput (Parallel)
Paradigm	General Purpose Programming	Matrix/Tensor Programming

What actually happens when you call `torch.matmul(A, B)` in Python?



Before optimizing our code, we must understand how memory is physically structured.

High Bandwidth Memory (HBM)

- **Scope:** Shared by all SMs on the device.
- **Capacity:** Massive (e.g., 80GB on H100).
- **Speed:** Relatively slow.
- **Usage:** Stores the model weights and the entire KV Cache.

Shared Memory (SRAM)

- **Scope:** Physically built onto a single SM.
- **Capacity:** Tiny (usually 100KB per SM).
- **Speed:** 20-30x faster than HBM.
- **Usage:** Used for intermediate matrix math to bypass total memory trips.

1. LOAD

Small chunks from HBM



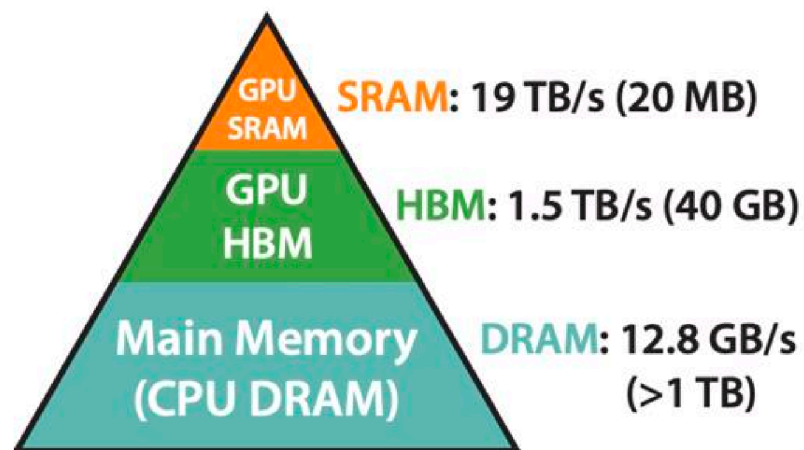
2. COMPUTE

Rapidly inside SRAM



3. STORE

Results back to HBM



Memory Hierarchy with Bandwidth & Memory Size

GPUs execute millions of tasks at once by mapping to this hardware hierarchy:

1. Thread —

The smallest unit executing an instruction.

Memory: Maps to an individual data element.

2. Block □

A group of threads on one SM (Streaming Multiprocessor).

Memory: Threads within a block share fast SRAM.

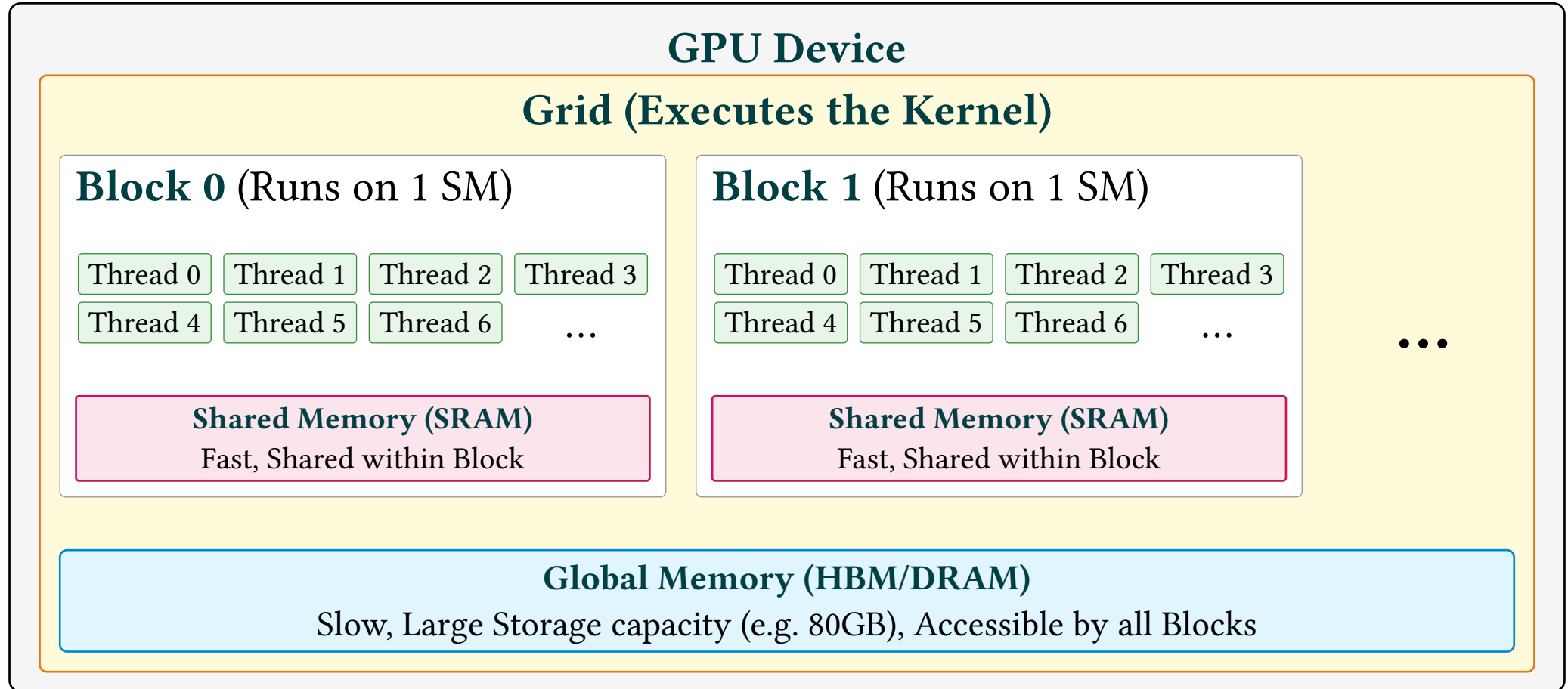
3. Grid ≡

A collection of blocks executing the entire algorithm/kernel job across the GPU.

Memory: Full access to HBM.

Example: Vector Addition of 1 Million elements

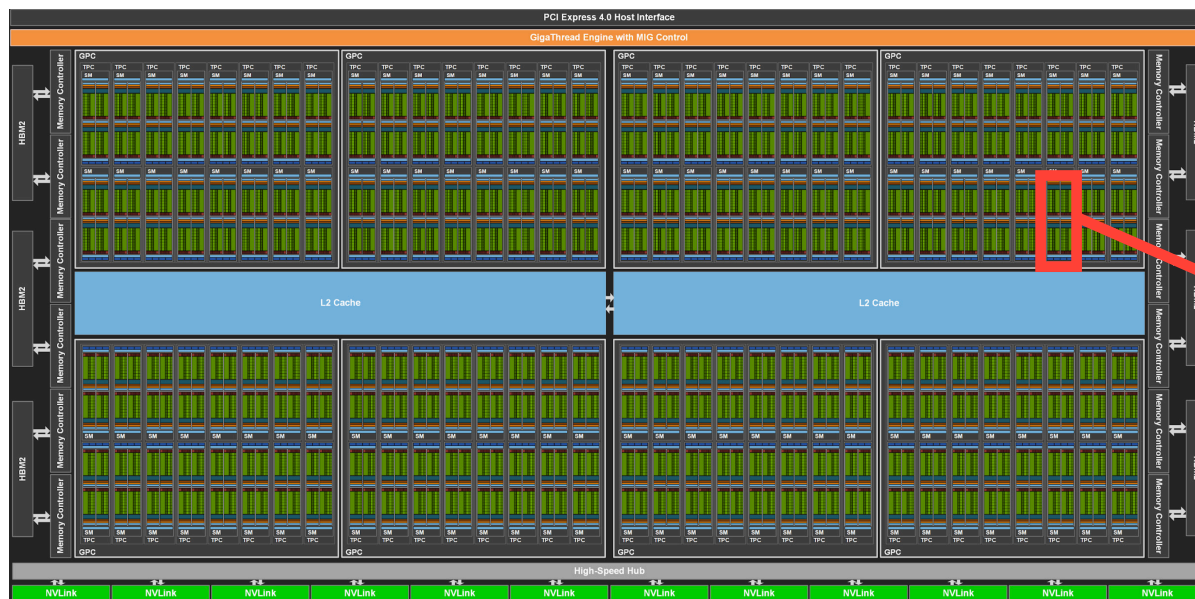
- Grid = 1000 Blocks, process all 1M elements
- Block = 1000 Threads, process 1000 elements each
- Totally we have $1000 \times 1000 = 1M$ threads, and each thread adds 1 element.



NVIDIA GA100 Chip Architecture



南方科技大学



Full GA100 Architecture

GA100 have 128 SMs that independently execute blocks (jobs)



SM (Streaming Multiprocessor)

SM have independent SRAM

Programming in Triton

Writing your first GPU computation kernel.

What is Triton?



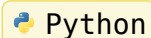
南方科技大学

Triton is a modern language and compiler for writing efficient GPU code.

1. Python-Based

Uses standard Python syntax.
Just add the decorator!

```
import triton
@triton.jit
def kernel(...):
```



2. Compiler

Triton's backend automatically compiles your Python code down to highly optimized PTX/SASS GPU machine code.

3. Block-Level

- CUDA targets **1 thread**.
- Triton targets **1 Block**.
- PyTorch targets **the Grid**.

The compiler handles how thread-level work!

Triton has been widely used in industry LLMs.

It is now the core backend for vLLM, PyTorch 2.0+ and many projects.

A Triton kernel doesn't receive standard PyTorch tensors with shape data; it receives raw memory pointers in DRAM.

```
1 import triton
2 import triton.language as tl
3
4 @triton.jit
5 def add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
6     # This function is executed by ONE BLOCK.
7     # This block processes `BLOCK_SIZE` elements.
8     pid = tl.program_id(axis=0) # Which block is this, 0, 1, 2, ...?
```

 Python

- `x_ptr`, `y_ptr`, `output_ptr`: Pointers to the start of input and output tensors in DRAM.
- `n_elements`: Total number of items to process (used for bounds checking).
- `BLOCK_SIZE`: Number of elements this block processes.
- `pid`: The Block ID (0, 1, 2, ...) assigned to this specific execution block.

Kernel runs per block, we need to calculate which elements this block should process:

```
1  # Suppose BLOCK_SIZE = 1024
2  # Block 0 handles offsets [0 ... 1023] in the input array
3  # Block 1 handles offsets [1024 ... 2047] in the input array
4  # ...
5
6  block_start = pid * BLOCK_SIZE
7  offsets = block_start + tl.arange(0, BLOCK_SIZE)
8  # tl.arange(0, 1024) gives an array [0, 1, 2, ..., 1023]
```

 Python

Array Masking: What if `n_elements` is 1000? Block 0 will try to read up to offset 1023, causing a invalid memory access!

```
1  # Only process elements that are within the bounds of the input array
2  mask = offsets < n_elements
```

 Python

Loading, Compute, and Storing



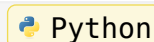
Load the data from DRAM arrays into on-chip SRAM, do the math, and write back.

```
1  # Load data from DRAM to SRAM (with safety mask!)
2  x = tl.load(x_ptr + offsets, mask=mask)
3  y = tl.load(y_ptr + offsets, mask=mask)
4
5  # Do the math (done automatically in registers/SRAM)
6  output = x + y
7
8  # Store back to DRAM
9  tl.store(output_ptr + offsets, output, mask=mask)
```

 Python

To run the kernel, we define **a grid (how many blocks to launch)** and call it from standard PyTorch.

```
1  def add(x: torch.Tensor, y: torch.Tensor):  
2      # First we need to allocate memory for the output on DRAM  
3      output = torch.empty_like(x)  
4      n_elements = output.numel()  
5      # Calculate grid size (ceiling division)  
6      # e.g., 2000 elements / 1024 = 2 blocks needed  
7      grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)  
8  
9      # Launch!  
10     # Choose block size (must be power of 32, for best performance)  
11     # The block size will be auto-tuned by Triton, 1024 is just a hint  
12     add_kernel[grid](x, y, output, n_elements, BLOCK_SIZE=1024)  
13     return output
```



Triton Auto-Tuning

Since the optimal `BLOCK_SIZE` is hardware-dependent, Triton's autotune will benchmark multiple configurations (e.g., 256 to 1024) to find the fastest one automatically at compile time.

Advanced Operator Fusion with Triton

Hitting the theoretical limit of memory bandwidth.

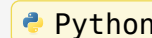
The Softmax Bottleneck



Stable Softmax: To prevent floating-point overflow, the numerically stable Softmax subtracts the maximum value $m = \max(x)$ before exponentiation:

$$y_i = \frac{\exp(x_i - m)}{\sum \exp(x_j - m)}$$

```
1  x_max = x.max(dim=1)      # [R] Read x
2  safe_x = x - x_max        # [R] Read x, [W] Write safe_x
3  num = torch.exp(safe_x)   # [R] Read safe_x, [W] Write num
4  denom = num.sum(dim=1)    # [R] Read num
5  return num / denom        # [R] Read num, [W] Write y
6                               # Total: 5 Reads + 3 Writes = 8 Accesses
```



Total Memory Operations: 8 HBM accesses for just one element (ignoring the much smaller `x_max` and `denom` loading).

Design Strategy: Row Parallelization



To fuse memory-bound ops, we must keep all intermediate data on-chip (SRAM).

However, Softmax need to read the entire row to compute the `max` / `sum`.

“Element-wise” Parallelism

Simple Vector Operation (e.g. Vector Addition)

- **Mapping:** 1 Block handles N Elements.
- **Problem:** Threads cannot easily communicate with neighbors.
- **Inefficiency:** Must write to DRAM to share intermediate results (Max/Sum).

“Row-wise” Parallelism

Fused Softmax To be Implemented

- **Mapping:** 1 **Block** handles 1 **Full Row**.
- **Advantage:** Threads within a block share high-speed **SRAM**.
- **Efficiency:** Compute Max/Sum on-chip, normalize, and store result once.

To implement fused softmax, we need to do row-wise parallelism.

Triton Strategy: Row-wise Partitioning



To implement Fused Softmax, we map each **Block** (Program) to a **Row** in the matrix.

```
1  @triton.jit
2  def softmax_kernel(output_ptr, input_ptr, stride, n_cols, BLOCK_SIZE: tl.constexpr):
3      # 1. Which row does this block process?
4      row_idx = tl.program_id(0)
5      # 2. Jump to the start of that row using 'stride'
6      row_start_ptr = input_ptr + row_idx * stride
7      # 3. Create pointer offsets for the entire row
8      col_offsets = tl.arange(0, BLOCK_SIZE)
9      input_ptrs = row_start_ptr + col_offsets
10     # 4. Handle boundaries and Load to SRAM
11     mask = col_offsets < n_cols
12     row = tl.load(input_ptrs, mask=mask, other=-float('inf'))
13     # --- Implement Softmax Logic ---
14     # ...
15     # --- Implement Store back to DRAM ---
16     # ...
```

Python

Lab 8 Exercises

GPU programming with Triton.

Task 1: Triton Vector Addition (`lab8_vector_add.py`) [70 points]

- Implement the `@triton.jit` kernel for mapping $Z = X + Y$.
- Compute pointer offsets using `tl.program_id` and `tl.arange`.
- Safely load arrays using `tl.load` with `mask`.
- Implement the PyTorch wrapper function and call the kernel using `[grid]`.

Task 2: Fused Softmax (`lab8_bonus_softmax.py`) [Bonus]

- Implement a fused Softmax to save DRAM roundtrips.
- Learn to manage 2D row-based parallelization and `tl.max` / `tl.sum` reductions.
- Prove you can match PyTorch's Softmax (CUDA) performance using Triton.

Thinking: Are there remained bottlenecks for Fused Softmax? (Hint: What problem does Online Softmax solve?)

Happy Coding!